

NPFL103: Information Retrieval (3)

Index construction, Index compression

Pavel Pecina

`pecina@ufal.mff.cuni.cz`

Institute of Formal and Applied Linguistics
Faculty of Mathematics and Physics
Charles University

Based on slides by Hinrich Schütze, University of Stuttgart.

Index construction

- Simple algorithms
- Distributed indexing
- Dynamic indexing

Index compression

- Motivation
- Term statistics
- Heaps' law
- Zipf's law

Index construction

RCV1 collection

- ▶ Shakespeare's collected works are not large enough for demonstrating many of the points in this course.
- ▶ As an example for applying scalable index construction algorithms, we will use the **Reuters RCV1** collection:
 - ▶ English newswire articles published in 1995–1996 (1 year).
 - ▶ Available from:
<https://trec.nist.gov/data/reuters/reuters.html>

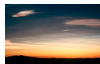
REUTERS

You are here: Home » Science » Article

[Science](#) [Business](#) [Lifestyle](#) [Technology](#) [Health](#) [Sports](#) [Entertainment](#) [Opinion](#) [Travel](#) [Environment](#) [Energy](#) [Weather](#) [Education](#) [Arts](#) [History](#) [Science](#)

Extreme conditions create rare Antarctic clouds

By Aug 1, 2009 6:01 PM

[Print This Article](#) [Print This Article](#) [Email This Article](#)

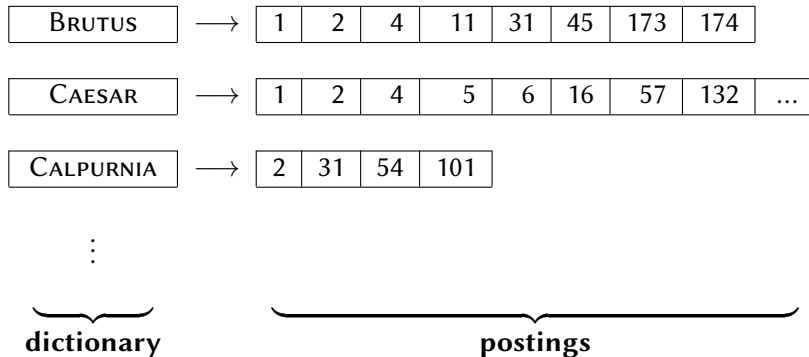
SYDNEY (Reuters) - Rare, mother-of-pearl colored clouds caused by extreme weather conditions above Antarctica are a possible indication of global warming, Australian scientists said on Tuesday.

Known as nacreous clouds, the spectacular formations glowing red and orange as the sun set were photographed by the sky over an Australian

Reuters RCV1 statistics

N	documents	800,000
L	tokens per document	200
M	terms (= word types)	400,000
	bytes per token (incl. spaces/punct.)	6
	bytes per token (without spaces/punct.)	4.5
	bytes per term (= word type)	7.5
T	non-positional postings	100,000,000

Goal: construct the inverted index



Index construction: Preprocess, generate postings, sort, create lists

Doc 1. I did enact Julius Caesar: I was killed i' the Capitol; Brutus killed me.

Doc 2. So let it be with Caesar. The noble Brutus hath told you Caesar was ambitious:

Doc 1. i did enact julius caesar i was killed i' the capitol brutus killed me

Doc 2. so let it be with caesar the noble brutus hath told you caesar was ambitious

term	docID	term	docID
i	1	ambitious	2
did	1	be	2
enact	1	brutus	1
julius	1	brutus	2
caesar	1	capitol	1
i	1	caesar	1
was	1	caesar	2
killed	1	caesar	2
i'	1	did	1
the	1	enact	1
capitol	1	hath	1
brutus	1	i	1
killed	1	i	1
me	1	i'	1
so	2	it	2
let	2	julius	1
it	2	killed	1
be	2	killed	1
with	2	let	2
caesar	2	me	1
the	2	noble	2
noble	2	so	2
brutus	2	the	1
hath	2	the	2
told	2	told	2
you	2	you	2
caesar	2	was	1
was	2	was	2
ambitious	2	with	2

term	doc. freq.	→	postings lists
ambitious	1	→	[2]
be	1	→	[2]
brutus	2	→	[1] → [2]
capitol	1	→	[1]
caesar	2	→	[1] → [2]
did	1	→	[1]
enact	1	→	[1]
hath	1	→	[2]
i	1	→	[1]
i'	1	→	[1]
it	1	→	[2]
julius	1	→	[1]
killed	1	→	[1]
let	1	→	[2]
me	1	→	[1]
noble	1	→	[2]
so	1	→	[2]
the	2	→	[1] → [2]
told	1	→	[2]
you	1	→	[2]
was	2	→	[1] → [2]
with	1	→	[2]

Sort-based index construction

- ▶ As we build index, we parse documents one at a time.
- ▶ The final postings for any term are incomplete until the end.
- ▶ Can we keep all postings in memory and then do the sort in-memory at the end?
- ▶ No, not for large collections
- ▶ At 10–12 bytes per postings entry, we need a lot of space for large collections.
- ▶ $T = 100,000,000$ in the case of RCV1: we can do this in memory on a typical current machine.
- ▶ In-memory index construction does not scale for large collections.
- ▶ Thus: We need to store intermediate results on disk.

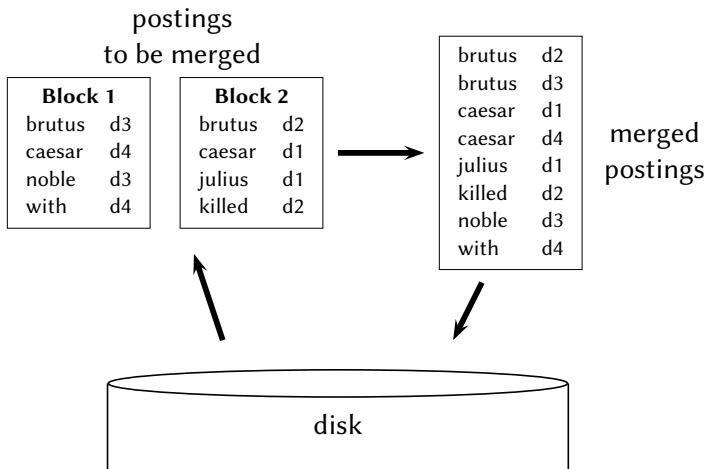
Same algorithm for disk?

- ▶ Can we use the same index construction algorithm for larger collections, but by using disk instead of memory?
- ▶ No: Sorting $T = 100,000,000$ records on disk is too slow – too many disk seek.
- ▶ We need an **external** sorting algorithm.

External sorting algorithm

- ▶ We must sort $T = 100,000,000$ non-positional postings.
 - ▶ Each posting has size 12 bytes (4+4+4: termID, docID, doc. freq).
- ▶ Define a **block** to consist of 10,000,000 such postings
 - ▶ We can easily fit that many postings into memory.
 - ▶ We will have 10 such blocks for RCV1.
- ▶ Basic idea of algorithm:
 - ▶ For each block:
 - (i) accumulate postings
 - (ii) sort in memory
 - (iii) write to disk
 - ▶ Then merge the blocks into one long sorted order.

Merging two blocks



Blocked Sort-Based Indexing (BSBI)

BSBIINDEXCONSTRUCTION()

```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4       $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5      BSBI-INVERT( $block$ )
6      WRITEBLOCKTODISK( $block, f_n$ )
7  MERGEBLOCKS( $f_1, \dots, f_n; f_{\text{merged}}$ )
```

► BSBI-INVERT:

1. sort [termID, docID] pairs
2. collect [termID, docID] pairs with the same termID

► Key decision: What is the size of one block?

Problem with sort-based algorithm

- ▶ Our assumption was: we can keep the dictionary in memory and use [termID, docID] postings.
- ▶ We need the dictionary (which grows dynamically) in order to implement a term to termID mapping.
- ▶ Actually, we could work with [term, docID] postings instead of [termID, docID] postings
- ▶ ...but then intermediate files become very large.
- ▶ ...We would end up with a scalable, but very slow index construction method.

Single-pass in-memory indexing (SPIMI)

- ▶ Key idea 1: Generate separate dictionaries for each block – no need to maintain term-termID mapping across blocks.
- ▶ Key idea 2: Don't sort. Accumulate postings in postings lists as they occur, sort later.
- ▶ With these two ideas we can generate a complete inverted index for each block.
- ▶ These separate indexes can then be merged into one big index.

SPIMI-Invert

```
SPIMI-INVERT(token_stream)
1  output_file  $\leftarrow$  NEWFILE()
2  dictionary  $\leftarrow$  NEWHASH()
3  while (free memory available)
4  do token  $\leftarrow$  next(token_stream)
5      if term(token)  $\notin$  dictionary
6          then postings_list  $\leftarrow$  ADDTODICTIONARY(dictionary,term(token))
7          else postings_list  $\leftarrow$  GETPOSTINGSLIST(dictionary,term(token))
8      if full(postings_list)
9          then postings_list  $\leftarrow$  DOUBLEPOSTINGSLIST(dictionary,term(token))
10     ADDTOPOSTINGSLIST(postings_list,docID(token))
11 sorted_terms  $\leftarrow$  SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms,dictionary,output_file)
13 return output_file
```

- ▶ Merging of blocks is analogous to BSBI.
- ▶ Compression of terms/postings makes SPIMI even more efficient

Distributed indexing

- ▶ For web-scale indexing: must use a distributed computer cluster
- ▶ Individual machines are fault-prone: can unpredictably slow down or fail.
- ▶ How do we exploit such a pool of machines?

Google data centers (estimates from 2016, Gartner)

- ▶ Google data centers mainly contain commodity machines.
- ▶ 2.5 million servers in 15 data centers are distributed all over the world.
- ▶ This was about 10% of the computing capacity of the world!

Exercise:

- ▶ If in a non-fault-tolerant system with 1000 nodes, each node has 99.9% uptime, what is the uptime of the system (assuming it does not tolerate failures)?

Answer: 37% ($0.999^{1000} = 0.3677$)

- ▶ Suppose a server will fail after 3 years. For an installation of 1 million servers, what is the interval between machine failures?

Answer: <2 minutes ($((3 * 365 * 24 * 60) / 1000000) = 1.5768$)

Distributed indexing

- ▶ Maintain a **master** machine directing the job – considered “safe”
- ▶ Break up indexing into sets of parallel tasks
- ▶ Master machine assigns each task to an idle machine from a pool.

Parallel tasks

- ▶ We will define two sets of parallel tasks and deploy two types of machines to solve them: **parsers**, **inverters**
- ▶ Break the input document collection into **splits** (corresponding to blocks in BSBI/SPIMI)
- ▶ Each split is a subset of documents.

Process

Master:

1. Assigns a split to an idle parser machine.

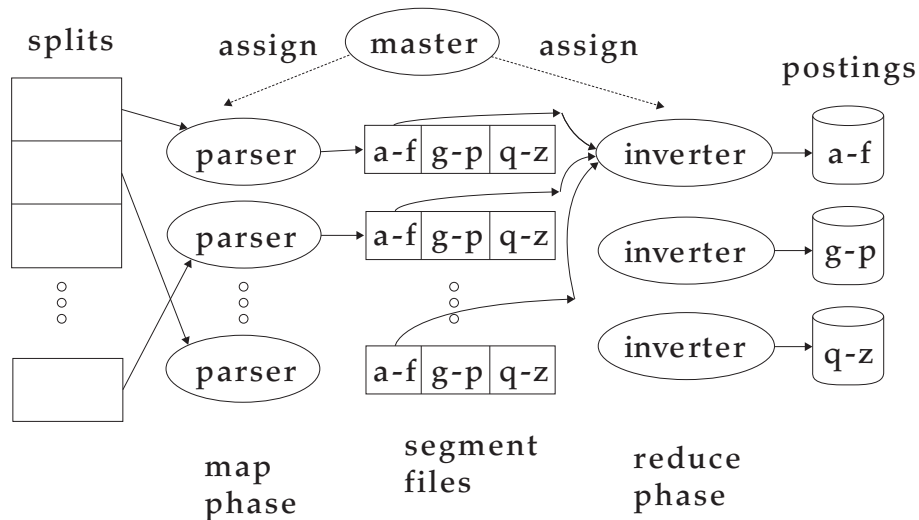
Parser:

1. Reads a document at a time and **emits** [term,docID]-pairs.
2. Writes pairs into j partitions each for a range of terms' first letters (e.g., a-f, g-p, q-z; here: $j = 3$).

Inverter:

1. Collects all [term,docID] pairs (= postings) for one term-partition (e.g., for a-f).
2. Sorts and writes to postings lists

Data flow



MapReduce

- ▶ The index construction algorithm we just described is an instance of MapReduce.
- ▶ MapReduce is a robust and conceptually simple framework for distributed computing ...
- ▶ ...without having to write code for the distribution part.
- ▶ The original Google indexing system consisted of a number of phases, each implemented in MapReduce.

Dynamic indexing

- ▶ Up to now, we have assumed that collections are **static**.
- ▶ They rarely are: Documents are inserted, deleted and modified.
- ▶ Dictionary and postings lists have to be **dynamically** modified.

Dynamic indexing: Simplest approach

- ▶ Maintain **big main index on disk**
- ▶ New docs go into **small auxiliary index in memory**.
- ▶ Search across both, merge results
- ▶ Periodically, merge auxiliary index into big index
- ▶ Deletions:
 - ▶ Invalidation bit-vector for deleted docs
 - ▶ Filter docs returned by index using this bit-vector

Issue with multiple indexes

- ▶ Corpus-wide statistics are hard to maintain.
- ▶ E.g., for hit-based spelling correction: how do we determine which correction has the most hits in the collection?
- ▶ We will see that other such statistics are important in ranking.
- ▶ There is no easy way around this if we want to do dynamic indexing efficiently.

Issue with auxiliary and main index

- ▶ Frequent merges
- ▶ Poor search performance during index merge
- ▶ Actually:
 - ▶ Merging of the auxiliary index into the main index is not that costly if we keep a separate file for each postings list.
 - ▶ But then we would need a lot of files – inefficient.
- ▶ Assumption for the rest of the lecture: The index is one big file.
- ▶ In reality: Use a scheme somewhere in between (e.g., split very large postings lists into several files, collect small postings lists in one file)

Logarithmic merge

- ▶ Logarithmic merging amortizes cost of merging indexes over time.
→ Users see smaller effect on response times.
- ▶ Maintain a series of indexes, each twice as large as the previous one.
- ▶ Keep smallest (Z_0) in memory
- ▶ Larger ones ($I_0, I_1, \dots$) on disk
- ▶ If Z_0 gets too big ($> n$), write to disk as I_0
... or merge with I_0 (if I_0 already exists) and write merger to I_1 etc.

LMERGEADDTOKEN(*indexes*, Z_0 , *token*)

```

1   $Z_0 \leftarrow \text{MERGE}(Z_0, \{token\})$ 
2  if  $|Z_0| = n$ 
3    then for  $i \leftarrow 0$  to  $\infty$ 
4      do if  $l_i \in indexes$ 
5        then  $Z_{i+1} \leftarrow \text{MERGE}(l_i, Z_i)$ 
6          ( $Z_{i+1}$  is a temporary index on disk.)
7           $indexes \leftarrow indexes - \{l_i\}$ 
8        else  $l_i \leftarrow Z_i$  ( $Z_i$  becomes the permanent index  $l_i$ .)
9           $indexes \leftarrow indexes \cup \{l_i\}$ 
10         BREAK
11      $Z_0 \leftarrow \emptyset$ 
```

LOGARITHMICMERGE()

```

1   $Z_0 \leftarrow \emptyset$  ( $Z_0$  is the in-memory index.)
2   $indexes \leftarrow \emptyset$ 
3  while true
4  do LMERGEADDTOKEN(indexes,  $Z_0$ , GETNEXTTOKEN())
```

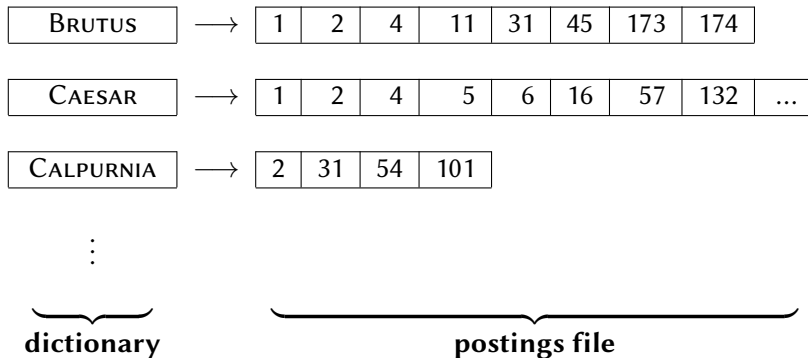
Logarithmic merge complexity

- ▶ Number of indexes bounded by $O(\log T)$ (T is total number of postings read so far)
- ▶ So query processing requires the merging of $O(\log T)$ indexes
- ▶ Time complexity of index construction is $O(T \log T)$.
...because each of T postings is merged $O(\log T)$ times.
- ▶ Auxiliary index: index construction time is $O(T^2)$ as each posting is touched in each merge.
 - ▶ Suppose auxiliary index has size a
 - ▶ $a + 2a + 3a + 4a + \dots + na = a \frac{n(n+1)}{2} = O(n^2)$
- ▶ So logarithmic merging is an order of magnitude more efficient.

Index compression

Inverted index

For each term t , we store a list of all documents that contain t .



- ▶ How much space do we need for the dictionary?
- ▶ How much space do we need for the postings file?
- ▶ How can we compress them?

Why compression? (in general)

- ▶ Use less disk space (saves money)
- ▶ Keep more stuff in memory (increases speed)
- ▶ Speed up transferring data from disk to memory (increases speed)

[read compressed data and decompress in memory]
is faster than
[read uncompressed data]

- ▶ Premise: Decompression algorithms are fast.
... this is true of the decompression algorithms we will use.

Why compression in information retrieval?

- ▶ First, we will consider space for dictionary
 - ▶ Main motivation for dictionary compression: make it small enough to keep in main memory
- ▶ Then for the postings file
 - ▶ Motivation: reduce disk space needed, decrease time needed to read from disk
 - ▶ Note: Large search engines keep significant part of postings in memory
- ▶ We will use various compression schemes for dictionary and postings.

Lossy vs. lossless compression

- ▶ Lossy compression: Discard some information
- ▶ Several of the preprocessing steps we frequently use can be viewed as lossy compression:
 - ▶ lowercasing, stop words removal, stemming, number elimination
- ▶ Lossless compression: All information is preserved.
 - ▶ What we mostly do in index compression

Model collection: The Reuters collection

symbol	statistic	value
N	documents	800,000
L	avg. # word tokens per document	200
M	word types	400,000
	avg. # bytes per word token (incl. spaces/punct.)	6
	avg. # bytes per word token (without spaces/punct.)	4.5
	avg. # bytes per word type	7.5
T	non-positional postings	100,000,000

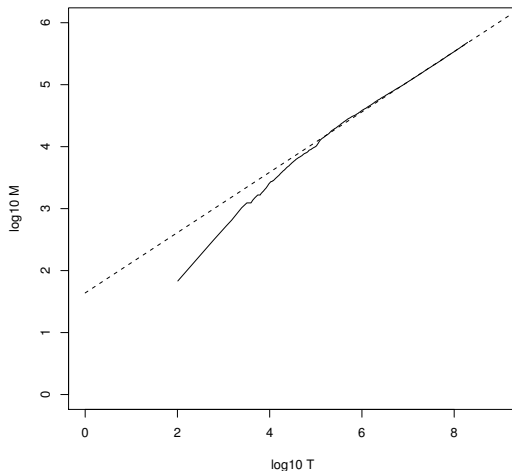
Effect of preprocessing for Reuters

size of	word types			non-positional postings			positional postings		
	dictionary			non-positional index			positional index		
	size	$\Delta\%$	$\sum \Delta\%$	size	Δ	$\sum \Delta\%$	size	Δ	$\sum \Delta\%$
unfiltered	484,494			109,971,179			197,879,290		
no numbers	473,723	-2%	-2%	100,680,242	-8%	-8%	179,158,204	-9%	-9%
case folding	391,523	-17%	-19%	96,969,056	-3%	-12%	179,158,204	-0%	-9%
30 stop w's	391,493	-0%	-19%	83,390,443	-14%	-24%	121,857,825	-31%	-38%
150 stop w's	391,373	-0%	-19%	67,001,847	-30%	-39%	94,516,599	-47%	-52%
stemming	322,383	-17%	-33%	63,812,300	-4%	-42%	94,516,599	-0%	-52%

How big is the term vocabulary?

- ▶ That is, how many distinct words are there?
- ▶ Can we assume there is an upper bound?
- ▶ Not really: At least $70^{20} \approx 10^{37}$ different words of length 20.
- ▶ The vocabulary will keep growing with collection size.
- ▶ Heaps' law: $M = kT^b$
 - ▶ Empirical law.
 - ▶ M – size of the vocabulary, T – number of tokens in the collection.
 - ▶ Linear in log-log space.
 - ▶ Typical values for the parameters: $30 \leq k \leq 100$ and $b \approx 0.5$.

Heaps' law for Reuters



Vocabulary size M is a function of collection size T :

$$M = kT^b$$

The best least squares fit for Reuters RCV1:

$$\log_{10} M = 0.49 * \log_{10} T + 1.64$$

$$M = 10^{1.64} T^{0.49}$$

$$k = 10^{1.64} \approx 44$$

$$b = 0.49.$$

Empirical fit for Reuters

- ▶ Good, as we just saw in the graph.
- ▶ For the first 1,000,020 tokens Heaps'law predicts 38,323 terms:

$$44 \times 1,000,020^{0.49} \approx 38,323$$

- ▶ The actual number is 38,365 terms, very close to the prediction.
- ▶ Empirical observation: fit is good in general.

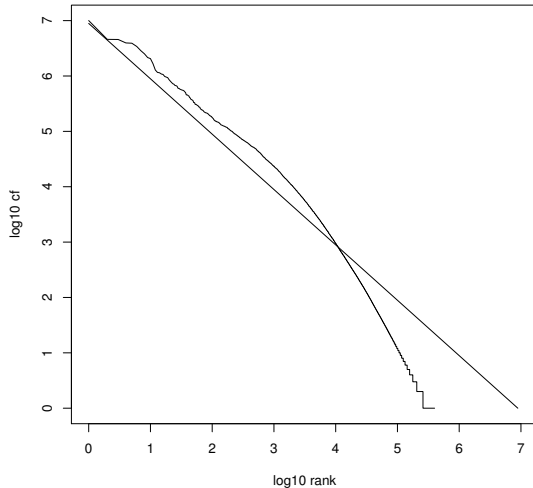
Zipf's law

- ▶ We have characterized the growth of the **vocabulary** in collections.
- ▶ We also want to know how many **frequent** vs. **infrequent terms** we should expect in a collection.
- ▶ In natural language, there are **a few very frequent terms** and **very many very rare terms**.
- ▶ Zipf's law: $cf_i \propto \frac{1}{i}$
 - ▶ The i^{th} most frequent term has frequency cf_i proportional to $1/i$.
 - ▶ Collection frequency cf_i : number of occurrences of term t_i in the collection.

Zipf's law: example

- ▶ Zipf's law: $cf_i \propto \frac{1}{i}$
- ▶ The i^{th} most frequent term has frequency cf_i proportional to $1/i$.
- ▶ So if the most frequent term (*the*) occurs cf_1 times, then the second most frequent term (*of*) has half as many occurrences $cf_2 = \frac{1}{2}cf_1 \dots$
- ▶ ...and the third most frequent term (*and*) has a third as many occurrences $cf_3 = \frac{1}{3}cf_1$ etc.
- ▶ Equivalent: $cf_i = ci^k$ and $\log cf_i = \log c + k \log i$ (for $k = -1$)
- ▶ Example of a power law.

Zipf's law for Reuters



Fit is not great. What is important is the key insight:

Few frequent terms, many rare terms.

Outline

Index construction

- Simple algorithms
- Distributed indexing
- Dynamic indexing

Index compression

- Motivation
- Term statistics
- Heaps' law
- Zipf's law